经过上一篇的简单了解，我们知道了虚拟机是怎么执行的，但是我们不知道里面是怎么进行数据交互的，这时候我们就需要学习 Lua 的数据结构，状态信息，调用信息，寄存器变化等等内容

首先就是重中之重的状态信息结构

```
struct lua_State
{
    CommonHeader;
    lu_byte allowhook;                          /* 是否允许触发钩子 */
    TStatus status;                             /* 线程当前状态 */
    StkIdRel top;                               /* 栈中第一个空闲槽位 */
    struct global_State *l_G;                   /* 指向全局状态 */
    CallInfo *ci;                               /* 当前函数的调用信息 */
    StkIdRel stack_last;                        /* 栈的末尾（最后元素+1） */
    StkIdRel stack;                             /* 栈的基地址 */
    UpVal *openupval;                           /* 当前栈中打开的上值链表 */
    StkIdRel tbclist;                           /* 待关闭变量链表 */
    GCObject *gclist;                           /* 垃圾回收链表 */
    struct lua_State *twups;                    /* 拥有打开上值的线程链表 */
    struct lua_longjmp *errorJmp;               /* 当前错误恢复点 */
    CallInfo base_ci;                           /* 第一层（C宿主）的调用信息 */
    volatile lua_Hook hook;                     /* 当前设置的钩子函数 */
    ptrdiff_t errfunc;                          /* 当前错误处理函数（栈索引） */
    l_uint32 nCcalls;                           /* 嵌套的不可挂起或C调用次数 */
    int oldpc;                                  /* 上次追踪的程序计数器值 */
    int nci;                                    /* 'ci'列表中的项数 */
    int basehookcount;                          /* 钩子计数基准值 */
    int hookcount;                              /* 当前钩子计数器 */
    volatile l_signalT hookmask;                /* 钩子掩码，控制触发事件 */
    struct
    {                                            /* 传输值信息（用于调用/返回钩子） */
        int ftransfer;                          /* 第一个传输值的偏移 */
        int ntransfer;                          /* 传输值的数量 */
    } transferinfo;
};
```

不过，里面还有一些嵌套的结构，我们等会会一一分析

2.CommonHeader

所有可回收对象的公共头，它的代码

```
#define CommonHeader            \
    struct GCObject *next; \
    lu_byte tt;            \
    lu_byte marked

typedef struct GCObject
{
    CommonHeader;
} GCObject;
```

就是这个东西，一个链表结构，展开以后就很好懂了

- `next`：指向下一个垃圾回收对象的指针
- `tt`：对象类型（对于线程是 `LUA_VTHREAD`）
- `marked`：垃圾回收标记位

```
typedef struct GCObject
{
    struct GCObject *next;
    lu_byte tt;
    lu_byte marked;
} GCObject;
```

2.lu_byte allowhook

```
typedef unsigned char lu_byte; // 对此数据结构的定义在这里
```

当前执行状态是否允许触发调试，控制调试用的 hook 能不能触发，这个在 debug 的时候用的多点，具体内容看见我们再结合上下文分析

3. TStatus status

```
typedef lu_byte TStatus;
```

线程当前状态，告诉你这线程在干嘛，它们的定义是这样的

```
/* thread status */
#define LUA_OK 0
#define LUA_YIELD 1
#define LUA_ERRRUN 2
#define LUA_ERRSYNTAX 3
#define LUA_ERRMEM 4
#define LUA_ERRERR 5
```

4.StkIdRel top，stack_last， stack

- `stack`：数组首地址（固定）
- `top`：当前可 push 位置（动态）
- `stack_last`：数组末尾/容量边界（固定）

```
typedef union
{
    StkId p; /* actual pointer */
    ptrdiff_t offset; /* used while the stack is being reallocated */
} StkIdRel;
```

栈之间的关系是

```
stack           top                 stack_last
|---------------|------------------|
栈底              栈顶（下一个空位）  栈容量上限
```

StkId

```c
typedef union Value
{
    struct GCObject *gc; /* GC 管理的对象：table、string、closure 等 */
    void *p;             /* light userdata，轻量 C 指针，不被 GC 管理 */
    lua_CFunction f;     /* light C 函数指针 */
    lua_Integer i;       /* Lua 整数 */
    lua_Number n;        /* Lua 浮点数 */
    lu_byte ub;          /* 占位，未使用，但可避免未初始化警告 */
} Value;

/*
** Tagged Values. This is the basic representation of values in Lua:
** an actual value plus a tag with its type.
*/

#define TValuefields  \
    Value value_; \ // 这个就是真正的值
    lu_byte tt_ // 这个是值的类型，联合体每次只能使用一种内部包含的数据类型

typedef struct TValue
{
    TValuefields;
} TValue;

typedef union StackValue
{
    TValue val; //“日常工作用”，几乎所有指令都用它
    struct
    {
        TValuefields;
        unsigned short delta; /* 内部用，Lua 编译器/GC 管理闭包列表时使用 */
    } tbclist; // “特殊场景”，Lua 内部用来管理 upvalue/闭包链
} StackValue;

/* index to stack elements */
typedef StackValue *StkId;  /* 栈指针，指向栈上的某个槽位 */
```

这个就是栈内的值，栈就像一个书架，每个格子是一个 StackValue（信封+标签），StkId 就是指向某个格子的手指。

VM 的指令 RA(i)/RB(i) 就是用手指去拿或放值

```
栈顶
┌──────────────┐
| StackValue   | val.value_ → 42
|              | val.tt_    → LUA_TNUMBER
└──────────────┘ <- StkId
| StackValue   | ...
| ...          |
```

5.struct global_State *l_G

指向全局状态的指针，所有线程共享的东西都在这里，字符串表、注册表、GC 状态、元表等，这个东西相当多了

```
typedef struct global_State
{
    lua_Alloc frealloc;   /* 内存分配函数，Lua VM 所有内存申请/释放/重分配都通过它 */
    void *ud;             /* frealloc 的辅助数据，一般指向内存管理器或状态 */
    l_mem GCtotalbytes;   /* 当前已经分配的内存总量 + GC欠债 */
    l_mem GCdebt;         /* GC待回收的“债务”，用于增量回收触发判断 */
    l_mem GCmarked;       /* 当前GC周期已标记的对象数量 */
    l_mem GCmajorminor;   /* 辅助计数，用于控制主/次垃圾回收切换 */

    stringtable strt;     /* 全局字符串哈希表，保证相同内容字符串唯一存储 */
    TValue l_registry;    /* Lua注册表，全局变量和C函数引用都存这里 */
    TValue nilvalue;      /* 内部固定的 nil 值 */

    unsigned int seed;    /* 随机种子，用于字符串哈希随机化 */

    lu_byte gcparams[LUA_GCPN]; /* GC参数表 */
    lu_byte currentwhite;        /* 当前白色对象标记，用于三色标记法 */
    lu_byte gcstate;             /* GC状态机，控制增量回收阶段 */
    lu_byte gckind;              /* 当前GC模式（普通/分代等） */
    lu_byte gcstopem;            /* 是否停止紧急回收 */
    lu_byte gcstp;               /* 是否暂停GC运行 */
    lu_byte gcemergency;         /* 是否为紧急回收状态 */

    GCObject *allgc;        /* 所有可回收对象链表 */
    GCObject **sweepgc;     /* 增量回收当前扫描的位置 */
    GCObject *finobj;       /* 带finalizer的对象链表 */
```

```
    GCObject *gray;          /* 灰色对象链表，等待标记子对象 */
    GCObject *grayagain;     /* 需要再次遍历的灰色对象 */
    GCObject *weak;          /* 值弱表对象链表 */
    GCObject *ephemeron;     /* key弱表（弱键表）链表 */
    GCObject *allweak;       /* 所有弱表对象链表 */
    GCObject *tobefnz;       /* 待回收的userdata链表 */
    GCObject *fixedgc;       /* 永不回收对象链表 */

    /* 分代GC相关字段 */
    GCObject *survival;      /* 活过一次GC的新生代对象 */
    GCObject *old1;          /* 老对象链表 */
    GCObject *reallyold;     /* 超老对象链表 */
    GCObject *firstold1;     /* old1链表中的第一个对象 */
    GCObject *finobjsur;     /* 活过一次的带finalizer对象 */
    GCObject *finobjold1;    /* old1带finalizer对象 */
    GCObject *finobjrold;    /* 超老带finalizer对象 */

    struct lua_State *twups; /* 拥有打开upvalue的线程链表 */

    lua_CFunction panic;     /* 出错时调用的C函数 */
    TString *memerrmsg;      /* 内存分配失败时的错误信息 */

    TString *tmname[TM_N];           /* 元方法名称数组，如 __index、__add 等 */
    struct Table *mt[LUA_NUMTYPES];  /* 基本类型的元表数组 */

    TString *strcache[STRCACHE_N][STRCACHE_M]; /* API用的短字符串缓存 */

    lua_WarnFunction warnf; /* Lua警告回调函数 */
    void *ud_warn;           /* warnf的辅助数据 */

    LX mainth;               /* Lua状态的主线程 */
} global_State;
```

里面对于语言的术语过于多了，到使用的地方我们再结合上下文来讲解，干讲对我和各位都很难，也很费时间

6.CallInfo *ci

当前调用信息，跟踪当前在执行的函数，函数在栈中的位置、栈顶、调用状态等，就是函数调用的"案发现场"记录

```
struct CallInfo
{
    StkIdRel func; /* function index in the stack 栈中函数的位置 */
    StkIdRel top; /* top for this function 本函数允许使用的栈顶位置 */
    struct CallInfo *previous, *next; /* dynamic call link 动态调用链 */
    union
    {
        struct
        { /* only for Lua functions */
            const Instruction *savedpc; /* 保存当前执行的PC指针 */
            volatile l_signalT trap; /* function is tracing lines/counts 调试或
计数时用  */
            int nextraargs; /* # of extra arguments in vararg functions vararg
函数多出的参数数量 */
        } l;
        struct
        { /* only for C functions */
            lua_KFunction k; /* continuation in case of yields continuation，用
于yield后的恢复 */
            ptrdiff_t old_errfunc;  /* 保存原来的错误处理函数位置 */
            lua_KContext ctx; /* context info. in case of yields
yield/continuation的上下文 */
        } c;
    } u;
    union
    {
        int funcidx; /* called-function index 调用的函数在栈上的索引  */
        int nyield; /* number of values yielded 该帧 yield 返回的值数量 */
        int nres; /* number of values returned 返回给上一层调用的结果数量 */
    } u2;
    l_uint32 callstatus; // 快速判断函数类型和状态
};
```

7. *UpVal openupval

这是当前栈上"打开"的上值（upvalue）链表头。Lua 的闭包可以捕获外部局部变量，这些变量成为上值（upvalue）。

```
typedef struct UpVal {
    CommonHeader;
```

```
    union {
        TValue *p;          /* 指向栈上的位置（当upvalue打开时） */
        ptrdiff_t offset;   /* 栈重分配时使用 */
    } v;
    union {
        struct {            /* 当upvalue打开时 */
            struct UpVal *next;   /* 链表中下一个 */
            struct UpVal **previous;
        } open;
        TValue value;       /* 当upvalue关闭时的值 */
    } u;
} UpVal;
```

- **打开的上值**：当闭包访问的外部局部变量仍在栈上活跃时，upvalue 直接指向栈位置
- **关闭的上值**：当外部函数返回后，局部变量从栈移除，值被"关闭"到堆上

8. StkIdRel tbclist

待关闭变量列表的栈指针。Lua 5.4 引入了 `to-be-closed` 变量（通过 `<close>` 元方法），这个指针指向栈上需要关闭的变量链表头。

9. *_GCObject _gclist__

用于垃圾回收的链表指针。线程本身也是可回收对象，通过这个字段链接到全局状态的 GC 链表中。

10. *_struct lua_State _twups__

拥有打开 upvalue 的线程链表。当线程有打开的上值时，会被链接到这个链表，以便 GC 能正确处理。

11. *_struct lua_longjmp _errorJmp__

当前错误恢复点的指针，用于实现错误处理机制（类似 C 的 setjmp/longjmp）。

12. CallInfo base_ci

第一层调用信息（C 宿主调用）。这是调用链的起点，当 Lua 状态初始化时创建。

13. volatile lua_Hook hook

当前设置的调试钩子函数，在特定事件（调用、返回、执行指令等）发生时被调用。

14. ptrdiff_t errfunc

当前错误处理函数在栈中的索引。当发生错误时，Lua 会查找这个函数来处理。

15. l_uint32 nCcalls

嵌套的非可挂起调用或 C 调用的数量。低 16 位是递归调用计数，高 16 位是非可挂起调用计数。

16. int oldpc

上次追踪的 PC（程序计数器）值，用于调试和性能分析。

17. int nci

当前 CallInfo 列表中的项数，表示调用栈深度。

18. int basehookcount

钩子计数的基准值，控制钩子触发的频率。

19. int hookcount

当前钩子计数器，每条指令执行时递减，归零时触发钩子。

20. volatile l_signalT hookmask

钩子掩码，控制哪些事件会触发钩子（调用、返回、行、计数等）。

21. struct { int ftransfer; int ntransfer; } transferinfo

用于调用/返回钩子中传输值的信息：

- `ftransfer`：第一个传输值的偏移
- `ntransfer`：传输值的数量

小结

`lua_State` 是 Lua 虚拟机的 **心脏**，它整合了执行栈、调用链、上值、垃圾回收、错误处理与调试机制，为 Lua 提供了高效、灵活且安全的运行时环境。理解其结构是深入掌握 Lua 内部工作原理的关键一步。

为啥后面没写了，是因为给我写吐了，太 TM 的难写了，我自己分析的时候是真没想到这么多